CITS 3002
Computer Networks

Lab 4: Wireless and Ethernet

Note: This lab runs for multiple weeks, so don't panic if you don't finish in the first week.

Worth: 2% of the unit

Please note: ask your lab demonstrator about any questions you have regarding previous labs. It will be beneficial for you to have a good foundation early on in the semester, and we are here to help!

In this lab, we will cover wireless and Ethernet related tasks, such as flooding and collision detection.

1. Tasks

TASK 1. FLOODING

The `protocol` in `etherflood.py` and its corresponding topology file `ETHERFLOOD` simply write IEEE802.3 format frames to a single LAN segment (this is a shared medium, not switched). The frames are of random length, and written at random intervals with a requested maximum frequency (such as 1000usecs). No attempt is made to avoid or resolve collisions on the LAN segment and, thus, when frames are written at a high rate, collisions are observed.

When a node has attempted to send a frame and then observes a collision, it simply gives up on that frame and then generates a new frame. First, run the following command for various different maximum frequencies, and observe the different results. In this example the maximum frequency is 10000usec, you might like to try 100000, 10000, 1000, and even 100 (note that the running time of the simulation is inversely proportional to the maximum frequency, and 100usec may take several minutes).

```
python sim.py --execution-duration 90s --silent-nodes --stats-csv etherflood_10000.csv -S 62 ETHERFLOOD 10000
```

After gathering the results from different maximum frequencies, compare the total number of collisions from each run.

TASK 2. CSMA/CD

Next, take a copy of `etherflood.py` and develop a *more correct* IEEE802.3 implementation by employing the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) algorithm.

We will begin with the carrier sense only. Each node can sense if the LAN segment to which it is connected is busy, by calling `carrier_sense(linknum)`. Naturally, if a node detects a signal on the segment, it should not transmit its frames as they will cause a collision, instead it should wait until the medium is clear and then transmit.

Note that the `FRAMECOLLISION` and `PHYSICALREADY` events are raised upon completion and decoding of the incoming signals, so waiting until the medium is clear can effectively be done by waiting until we receive either of these events. This infrastructure has already been set up for you in etherflood.py (see the calls to `schedule_send_on_clear()`), so to implement the carrier sense you will only need to modify the `send_frame` function.

When your carrier sense is implemented, try running the simulations again using the same parameters (remember to update your `ETHERFLOOD` topology file to point to your newly modified protocol `.py` file):

```
python sim.py --execution-duration 90s --silent-nodes --stats-csv csma_10000.csv -S 62 ETHERFLOOD 10000
```

TASK 3. EXPONENTIAL BACKOFF

Next we will implement the collision detection and binary exponential backoff. If you look at the definition of the `collision` method, you will see that nodes that have sent a frame and then detect a collision will respond by giving up on the frame and scheduling generation of a new message. This is obviously not an ideal implementation!

Replace the offending section of code with binary exponential backoff. The number of observed collisions for the current frame should be increased, the maximum number of backoff slots calculated as min(1024, 2^collisions) - 1, a random number of backoff slots chosen, and then an attempt to retransmit the frame is scheduled.

Do not implement the IEE802.3 upper limit of 16 collisions. Instead, continue rescheduling with a maximum backoff of 1023 slots indefinitely.

Note that only one section of code needs to be changed. The other pieces, such as resetting the collision counter on a successful send, are already in place. When the implementation is complete, run the simulations again:

```
python sim.py --execution-duration 90s --silent-nodes --stats-csv csmacd_10000.csv -S 62 ETHERFLOOD 10000
```

2. Quiz on LMS

To gain 2% allocated mark for this lab, please ensure that you solve lab questions on LMS by the deadline.